

Single source of truth · supersedes all earlier plans

TapPay implementation plan

Security remediation, App Store submission and Xero integration in one ordered sequence. Every file path, line number and count below was read from origin/feat/tablet-desktop-app @ 9a51bd8. Written to be executed by someone who cannot infer this codebase's unwritten conventions.

Baseline 9a51bd8Routes 215No live merchants — no incident clock

What is and is not decomposed here — read this first

with exact locations and acceptance checks. That is every piece of work that can start in the next several weeks.

That is deliberate, not an omission: R3's tasks depend on what R2's provider boundary actually looks like once built, and inventing that detail now would produce confident-sounding instructions that turn out wrong. Each phase gets decomposed the same way once its predecessor lands. Part 5 says exactly what each contains so nothing is lost in the meantime.

Contents

P0	How to use this document
P1	The map and sequence
P1.2	Verified current state
P2	Ground rules
P2.2	Config & response contracts

Executable now

P3	R0 — containment (11 tasks)
----	-----------------------------

P4R1 — harness & auth (12 tasks)

P6Apple track (9 tasks)

P7Xero track (12 tasks)

Scoped, decomposed later

P5R2–R8 scope & gates

Reference

P8.1Evidence & command gate

P8.2Commit sequence

P8.3Stop conditions

P8.4Explicitly deferred

P8.5Review & handoff templates

P8.6Final go-live gate

P9Decisions register

P10Verification record

P0

Work top to bottom within a phase. Never start a task whose dependencies are unmet. Never mark a task done without its acceptance check passing.

Every task carries an ID, an owner badge — **Agent** means a coding model may do it, **You** means it requires a human and must not be delegated — a dependency line, the work itself with exact file locations, and a green acceptance check. The check is the definition of done. If it does not pass, the task is not done, and weakening the check to make it pass is a stop condition.

1. Where a task says to prove a defect before fixing it, that ordering is mandatory. A fix with no prior failing test has no evidence it fixed anything, and the evidence is unrecoverable once the fix lands.
2. If this document does not say how something is done, find an existing example in the codebase and copy it. Part 2 lists the four places where the obvious approach is wrong here.
3. `git add -A` `git add .` Stage explicit paths, always excluding `.claude-home/**` and `.claude/settings.local.json`.
4. — not in commits, tests, evidence, screenshots or logs. Record rule ID, file and status only.
5. For code whose only implementation is fake, a feature flag around it is not containment.
6. Add a forward migration.
7. for a merchant who has not opted into the feature being built.
8. (P8.3). Leave payment initiation disabled, preserve evidence, escalate. Do not mark a phase complete because time ran out — record it incomplete with the exact blocker.

Things an agent must never do in this project

- Rotate production secrets, alter Replit Secrets, or create/delete Neon branches.
- Apply production migrations, or deploy without the named human approval.
- Rewrite Git history, delete retained financial data, or contact customers.
- Call live Windcave, or enable any capability flag in staging or production.
- Resolve a product, legal or provider decision by guessing. Build fail-closed scaffolding and tests while the feature stays disabled instead.

P1

Three workstreams. They are not parallel — remediation gates the other two, for a specific reason rather than a procedural one.

Why Xero cannot go first

Xero sync writes a merchant's revenue into their real, GST-filed accounting ledger. The app can currently mark a payment successful without any provider ever being

called — an unauthenticated request to `/api/payments/apple-pay/process` returns fabricated success whenever Windcave credentials are absent. Connecting Xero on top of that would propagate invented revenue into a tax return. The remediation plan's own feature freeze says the same thing in policy terms; this is the reason behind it.

NOW

Rotate committed secrets, kill fake-approval routes, tombstone the cross-tenant delete, make production seeding impossible. Nothing else starts first.

+ NOW

Both have zero code dependency on remediation. The Apple config items touch no payment code; Xero Phase 0 is portal registration and two questions to Xero support whose answers take days. Run both in parallel from day one.

AFTER R0

Includes the OAuth rebuild. Sign in with Apple is built here, not separately.

AFTER R1

Typed transport, exact URL validation, the shared outcome verifier.

AFTER R2

Notification inbox, one finaliser, aggregate state machines.
— payments become truthful.

AFTER R3

Xero mirrors payments and refunds; both must be provably correct first.

AFTER R4

Xero consumes R5's encryption envelope rather than building its own.

AFTER R8

Real money, Tap to Pay and live Xero sync. P8.6.

1. An earlier assessment marked the wallet buttons compliant — true of the buttons, but it never examined the routes behind them. `routes.ts:6203` and `:6266` have no auth middleware and fabricate success. Correct rating is blocking, and it belongs to R0 rather than the Apple track.
2. `src/plugins/TapToPayPlugin.swift` sits outside the iOS app target, contains no SDK integration — its nine Windcave references are all comments — and returns `STUB_TOKEN_<timestamp>`. It does not reach hardware. R7 owns the reality gate.
3. R5's key-ID envelope serves both legacy bank fields and Xero OAuth tokens. A simpler key-less format cannot rotate.
4. Google (fix), Apple (add), Xero (add). The existing Google route is currently the wrong pattern to copy — see R1-T4.

P1.2

Read directly from `9a51bd8`. These are the defects the plan exists to close.

Defect	Location	Fixed by
Operational secrets in tracked config	<code>.replit</code> lines 72-77	R0-T7 + H2
Cross-tenant destructive delete, no ownership check	<code>routes.ts:5517</code>	R0-T4
Production seeds <code>demo@tap.t.co.nz / demo123</code>	<code>index.ts:200, seed.ts:40</code>	R0-T4
Wallet routes: no auth, fabricate success	<code>routes.ts:6203, 6266</code>	R0-T5
Missing credentials select simulation	<code>5 × isConfigured() sites</code>	R0-T5
Public <code>?sim=1</code> forces simulated success	<code>routes.ts</code> callbacks	R0-T5
Startup dumps dev <i>and</i> production databases	<code>index.ts</code> ~216-232	R0-T6
Deployment build runs migrations	<code>.replit</code> build	R0-T6
OAuth: no state/nonce/PKCE; JWT in URL query	<code>routes.ts:336-351, 433-440</code>	R1-T4
71 permissive <code>parseInt(req.params</code>	<code>routes.ts</code>	R1-T5
116 authenticated routes, only 35 ownership checks	<code>routes.ts</code>	R1-T6
Test asserts a real hook-order crash	<code>zz-review-hooks-repro.test.tsx</code>	R1-T8
No <code>/healthz</code> or <code>/readyz</code>	—	R6
Tap to Pay stub returns STUB_TOKEN	<code>TapToPayPlugin.swift:95</code>	R0-T5 (off) / R7

P2

Four conventions where the obvious approach is wrong here

1. `server/storage.ts`
`IStorage` has 166 methods, implemented by `MemStorage` (in-memory, dev default) and `DatabaseStorage` (Drizzle, production). Every new database operation needs *three* edits: the interface signature, a `MemStorage` stub, and the real `DatabaseStorage` implementation. Never import `db` and query directly from a route or worker. For `MemStorage` stubs follow the existing precedent — writes `throw new Error("<feature> requires database")`, list reads return `[]`, single reads return `undefined`. Never make `MemStorage` half-work.
 2. `server/routes.ts` exports one `registerRoutes(app: Express)` containing all 215 registrations. Do not create `server/routes/<feature>.ts` — there is no precedent. Add routes inside `registerRoutes` under a clear comment banner, keeping business logic in a separate module so handlers stay thin.
 3. Add to it; do not create a second import statement.

```
import { pgTable, text, serial, decimal, timestamp, boolean, integer, jsonb, uuid, uniqueIndex, index } from "drizzle-orm/pg-core";
```
 4. `--force` so new nullable columns and new tables apply themselves while destructive changes are refused. That is a deliberate safety property. If you ever want `--force` you have made a mistake — stop and report it.
- Never `git add -A` or `git add .` — explicit paths only, always excluding `.claude-home/**` and `.claude/settings.local.json`.
 - Never edit an already-applied migration. Add a forward migration.
 - Never delete historical financial rows to make a test pass.
 - Never refresh a screenshot baseline, allowlist or expected error to turn a verifier green. Visual baseline changes need product approval.
 - Never put secret values, bank data, raw payment tokens, processor session IDs, return state, notification bodies or customer PII into evidence or commit messages.
 - Preserve unrelated dirty work; do not sweep it into a remediation commit.

A task is complete only when all of these hold. A TODO, stub, simulator or documentation promise is not an implementation.

- The implementation exists.
- A failing test was demonstrated, or the pre-change defect was otherwise captured.
- Positive, negative, retry and no-side-effect tests pass as applicable.
- Any schema- or data-changing migration was rehearsed on an isolated restored snapshot.
- Logs and API responses expose no sensitive values.
- Phone, tablet and desktop regression checks pass when client code changed.
- The phase evidence record names the commit, commands, results, remaining risks, rollback point and human approvals.

Evidence lives at `docs/evidence/remediation-v2-2/<phase>/`. Commit sanitised text reports and small approved screenshots only — never database dumps, provider payloads, user invoices, trace archives containing tokens, or raw production exports.

- `docs/decisions/` No new vertical, payment method, native capability, redesign or growth feature. Only incident containment, correctness and security fixes, required compatibility work, tests, operations and truthful documentation. Scope expansion needs a recorded product/security decision.
- `docs/decisions/` holds one immutable dated ADR per decision, schema choice, retention choice or exception. Never rewrite an accepted ADR — supersede it.
- A reviewer completes the P8.5 template against the proposed phase; a second reviewer confirms every blocking finding is resolved. No implementation begins until the recommendation is *Approve*. "Approve with unresolved blockers" is not approval.
- — small typed services, explicit states, database constraints, observable retries.
- — invalid target, config, state, identity, provider response or migration stops before side effects.
- — authenticate, authorise, validate and derive tenant, money and provider identity on the server.

- — every initiation, transition, refund, reconciliation and manual correction has an immutable local record and one accountable actor.
- — expansion/compatibility/migration/contract releases, bounded locks, tested rollback. "Reversible" never means restoring a known vulnerability or reverting encrypted rows to plaintext writes.

P2.2

Variable	Rule	Failure behaviour
APP_ENV	development · test · staging · production. Do not infer from <code>NODE_ENV</code> alone	Invalid/missing in production: startup failure
PUBLIC_ORIGIN	One canonical HTTPS origin; no path, query, fragment, credentials or wildcard	Startup failure for OAuth, reset, invite, payment or notification deployments
DATABASE_TARGET	local · ci · staging · production; must agree with the approved target	Mismatch stops work
DATABASE_URL	Nonempty; never logged	Startup/readiness failure
JWT_SECRET	Strong, newly rotated, independently generated	Startup failure in staging/production
PAYMENT_RETURN_STATE_SECRET	Independent of <code>JWT_SECRET</code>	Payment initiation disabled if absent; never fall back to JWT
CRON_SECRET	Strong secret when any scheduled money/message job is enabled	Readiness failure for enabled jobs
SEED_DEMO_DATA	Strict true/false; default false	<code>true</code> is fatal in staging/production

```

PAYMENT_MODE = disabled | simulation | uat | production

test          default disabled · allows disabled, simulation (isolated
transport only)
development   default disabled · allows disabled, simulation, uat (explicit
credentials)
staging       default disabled · allows disabled, uat
production    default disabled · allows disabled, production only

// invariants
missing credentials NEVER select simulation
simulation in staging/production → startup error
uat in production → startup error
production requires FEATURE_LIVE_WINDCAVE=true (defaults false)
a partial credential pair is an ERROR, not "not configured"
UAT requires the exact approved UAT endpoint + complete username/API-key pair
production requires the exact approved production endpoint + complete pair

```

Provider configuration is captured after validation, never as unchecked module-level strings at import time. Reconciliation of already-created legitimate attempts is controlled separately and stays available when new initiation is disabled.

```

FEATURE_NEW_RETAIL_PAYMENTS    FEATURE_SUBSCRIPTION_CHARGING
FEATURE_INVOICE_PAYMENTS       FEATURE_TAP_TO_PAY
FEATURE_SPLIT_CONFIGURATION     FEATURE_ECOMMERCE_API
FEATURE_REFUND_INITIATION       FEATURE_LIVE_WINDCAVE
FEATURE_XERO_SYNC               FEATURE_PROVIDER_RECONCILIATION // exception,
below

```

```

FEATURE_CRYPTO // false-only invariant: absent/false accepted, true = startup
error everywhere

```

`FEATURE_PROVIDER_RECONCILIATION` is the one flag that must remain *true* once real provider sessions exist, even during an initiation kill-switch — otherwise legitimate in-flight payments can never finalise. Only an incident commander stops it.

Skip the compatibility-alias layer

The source remediation plan describes aliasing legacy flag names

`FEATURE_REFUNDS`, `FEATURE_SPLIT_PAYMENTS`, `FEATURE_WOOCOMMERCE`, `ENABLE_PER_PAYMENT_LINKS`. Verified: `FEATURE_*` or `ENABLE_*` variables are parsed anywhere in `server/` today. There is no existing behaviour to stay compatible with, so build the canonical names directly and skip the alias layer entirely.

Every disabled-route test asserts the status *and* zero business-row mutation, provider call, success SSE, push, outbox entry, usage increment or refund/settlement change.

C o d e	Meaning
401	Authentication missing, invalid, expired or disabled
400	Authenticated request with malformed path/query/body input
403	Authenticated principal lacks role or tenant permission
404	Tenant-safe: identifier exists outside the caller's scope where disclosure would enumerate. Public: removed or nonexistent capability
409	Idempotency/state conflict, or operation already in progress
410	Deliberate compatibility tombstone (retired clearing, bank-data update)
422	Well-formed request violating a domain rule, where existing contracts permit it
502	Confirmed upstream failure with known retry/reconciliation semantics
503	Implemented capability temporarily disabled, or required provider mode unavailable — with a stable error code

```
2      Durable operation accepted, provider outcome pending or unknown
0
2
```

- 1. authenticate
- 2. strict parameter parse
- 3. capability / payment-mode check
- 4. role and tenant authorization
- 5. strict body / file validation
- 6. billing / entitlement check
- 7. atomic idempotency / state claim
- 8. provider call
- 9. atomic finalization
- 10. allowlisted DTO + event publication

Multipart parsing moves *after* authorisation and size/type prechecks. The logo upload route must not accept or write a file before merchant ownership is known.

Part 3 · Execute first · blocks everything

Eleven tasks: six an agent can do, five only a human can. They interleave, and the order matters more than the speed.

R0.0

Phase	Tasks	Why this order
A · Prepare	T0 → T1 → T2 → T3	Prove the holes, build the config spine, make VAPID rotation survivable
B · Fix code	T4 → T5 → T6	Close the holes while the T1 tests are still there to flip green
C · Secrets	T7 → H1 → H2 → H3 → H4 → H5	Scrub the tracked file <i>before</i> rotating

Two ordering traps that each cost a full redo

Do not set a new `JWT_SECRET` while it still lives in tracked `.replit`. Commit it and the new secret is burned exactly like the old one. T7 removes the block; only then does H2 generate fresh values straight into Replit Secrets.

Rotating the keypair invalidates every existing push subscription. Without T3 the client has no re-subscribe path and notifications simply stop, with no error anyone can see. T3 ships first.

You are not in an incident

No real money is flowing, so R0 is urgent rather than an emergency. That is a real advantage — you can rotate carefully, in the right order, with the client already able to survive it. Use it.

Phase A — Prepare

Nothing here changes production behaviour. It builds the evidence and the spine.

R0.A

R0-T0Agent

Cut the branch and record the base

Depends on: nothing · Blocks: everything

First confirm with the repo owner that `9a51bd8` is still the intended base and that none of the other 14 remote branches — `feat/trades-phase4-delivery-payments`, `feat/property-dashboard-redesign`, `replit-agent` and the rest — must merge first. Merging a large branch mid-remediation invalidates every piece of evidence collected up to that point.

```
git fetch origin
git checkout -b remediation/r0-containment 9a51bd8
git rev-parse HEAD # record in the handoff
git status --short # record pre-existing dirty paths
node --version; npm --version
npm ci
```

```
npm run check
npm test -- --runInBand      # characterisation only
```

```
npm run db:migrate:status    # NAMED NON-PRODUCTION TARGET ONLY
```

The suite passes *while containing*

`client/src/__tests__/zz-review-hooks-repro.test.tsx`, which asserts a real React hook-order crash. That is characterisation, not approval. Record it; it belongs to R1-T8. Also record any `act(...)` warnings and open handles.

Migration status should report 19 applied, 0 pending, 0 drifted, 0 orphaned. Anything else is a stop condition — do not blindly apply.

Check

Branch exists from the right SHA. `npm run check` passes. Baseline test count, warnings, hook-repro test and migration status all recorded in the handoff before any edit.

R0-T1AgentMust fail first

Write the failing tests that prove each hole

Depends on: T0 · Blocks: T4, T5, T6

New file `server/__tests__/r0-containment.test.ts`. Follow the existing patterns in that directory — `subscription-route-security.test.ts` and `phase0-contracts.test.ts` are the closest precedents. Jest config is `jest.server.config.cjs`: node environment, ts-jest, `@shared/*` mapped.

These must fail on 9a51bd8. If any passes immediately you have written it wrong — find out why before continuing.

Test	Asserts — and currently fails because...
------	--

<code>cross-tenant clear</code>	Merchant A calling <code>POST /api/merchants//clear-transactions</code> gets 403/404 and <code>clearTransactions</code> is never called — <code>routes.ts:5517</code> parses the ID and calls straight through with no ownership check
---------------------------------	---

wallet routes need auth	<code>POST /api/payments/apple-pay/process</code> unauthenticated is rejected — <code>routes.ts:6203</code> has no middleware at all
no simulation fallback	With credentials absent, no route returns success — <code>isSimulation = !windcaveService.isConfigured()</code> fabricates one
sim flag ignored	<code>?sim=1</code> <code>?SIM=1</code> repeated keys, encoded values and a body <code>sim</code> field all fail to change any outcome
production never seeds	With <code>APP_ENV=production</code> seeding does not run with the flag absent, false or true — <code>index.ts:200</code> is guarded only by <code>isDatabaseConnected()</code>
no demo login in production	<code>demo@tapt.co.nz</code> <code>demo123</code> cannot authenticate under production config
startup makes no backup	Booting in any mode never spawns <code>scripts/db-backup.sh</code> — <code>index.ts:~221</code> spawns it whenever <code>NODE_ENV !== 'production'</code>
no secrets in tracked config	Reading tracked <code>.replit</code> finds no <code>JWT_SECRET</code> <code>ADMIN_PASSWORD_HASH</code> or <code>VAPID_PRIVATE_KEY</code> assignment — currently lines 72–77

Check

`npm run test:server` shows these 8 failing. Paste the failure output into the handoff — it is the evidence the vulnerabilities were real, and the only chance to capture it. Once T4–T6 land it is unrecoverable.

R0–T2Agent

Fail-closed configuration module

Depends on: T0 · Blocks: T4, T5, T6

New file `server/config.ts`. Verified absent on this baseline — genuinely greenfield. Implement exactly the contract in P2.2: one immutable typed object, loaded before routes, Windcave

modules, seeding or cron. Production code reads `process.env` *only* here; tests construct it from an explicit object.

Parse booleans strictly — never JavaScript truthiness. Build canonical flag names directly; skip the alias layer (P2.2).

Check

- Unit test covers `undefined` empty string, `0` `false` `FALSE` `true` `TRUE` and junk for every boolean.
- Every invalid mode/environment pair throws at startup: `simulation` in staging or `production`, `uat` in production, `FEATURE_CRYPTO=true` anywhere, a partial credential pair.
- `npm run check` passes.

R0-T3AgentGates H2

Make VAPID rotation survivable

Depends on: T0 · Blocks: H2

Rotating VAPID invalidates every existing push subscription. Today the client has no recovery path, so after rotation notifications would silently stop. Fix that before the key changes.

The client must detect that its stored subscription no longer matches the server's current public key, discard it, and re-subscribe — with no merchant action and no visible error. Server-side, an endpoint that returns 404/410 from the push service should be pruned rather than retried forever.

Check

With a deliberately changed public key in a dev environment: an existing subscriber re-subscribes automatically on next load, receives a test push, and no error surfaces in the UI. Stale rows are pruned rather than accumulating.

Phase B — Fix the code

Flip the T1 tests green. Each task removes a capability rather than hiding it.

R0.B

R0-T4Agent

Tombstone cross-tenant clearing, lock down seeding

Depends on: T1, T2

Clearing — `server/routes.ts:5517`

Replace the body. It must never delete. Follow the middleware order from P2.2: authenticate, parse strictly, authorise, then respond.

```
unauthenticated → 401
authenticated, malformed ID → 400 // "1abc", "0", "-1", "1.5", arrays
member or different merchant → 403 or tenant-safe 404 per the policy
matrix
```

```
authenticated owner, own merchant → 410 TRANSACTION_CLEARING_RETIRED
```

After proving no call sites remain, remove `clearTransactions` from the `IStorage` interface and from both `MemStorage` and `DatabaseStorage` — three edits, per P2 convention 1. Do not replace it with an owner-authorised hard delete; financial corrections use typed transitions and append-only events.

Seeding — `server/index.ts:200` and `server/seed.ts`

Current guard is `isDatabaseConnected()` alone, so a fresh production database will seed `demo@tapt.co.nz / demo123`. Gate on `SEED_DEMO_DATA === true` and `APP_ENV` being development or test. Add a second independent assertion inside `seedDatabase()` so direct invocation also refuses staging and production.

Keep the existing "exit if merchants exist" guard — necessary but not sufficient, since a fresh production database has no merchants. If explicit local seeding fails, surface the failure rather than swallowing it into a log line.

Check

- The cross-tenant, production-seed and demo-login tests from T1 pass.
- Zero calls to `clearTransactions` zero transaction updates, zero provider/SSE/push/outbox effects on the tombstoned route.

- `npm run check` passes — if `MemStorage` still declares the removed method, tsc catches it, which is the point.
- App boots with no `DATABASE_URL` (`MemStorage` path) without crashing.

R0-T5AgentLargest task

Remove the fake-success surfaces

Depends on: T1, T2

Disposition is mandatory per route. A flag wrapped around code whose only implementation is fake is not containment.

Route	Line	Disposition
<code>/api/transactions/tap-to-pay</code>	22 10	Disable behind <code>FEATURE_TAP_TO_PAY=false</code> no DB mutation; no simulator outside isolated tests
<code>/api/merchants/:id/nfc-pay</code>	25 15	Disable behind <code>FEATURE_TAP_TO_PAY=false</code> no DB mutation
<code>/api/nfc-sessions/:id/complete</code>	25 95	Remove from production router; public 404; retire the <code>/nfc</code> simulator consumer
<code>/api/windcave/sim-submit</code>	30 67	Remove from production router; public 404; fixture lives only in isolated tests
<code>/api/payments/apple-pay/process</code>	62 03	Remove or tombstone. No success fallback under any condition
<code>/api/payments/google-pay/process</code>	62 66	Remove or tombstone. No success fallback under any condition
<code>callback ?sim=1</code>	—	Ignore and reject <code>sim</code> entirely; only a persisted provider attempt can reconcile
Refund initiation	—	<code>FEATURE_REFUND_INITIATION=false</code> → 503 <i>before</i> any balance reservation
Admin API-key / <code>/api/v1</code>	—	<code>FEATURE_ECOMMERCE_API=false</code> strip mock success data; public surface 404

Also remove the five `windcaveService.isConfigured()` simulation branches. Missing credentials must produce a disabled state, never a fabricated success.

Client side: hide the removed controls or show a truthful unavailable state in `client/src/App.tsx`, `pages/nfc-payment.tsx`, the terminal pages, `components/native-payment-buttons.tsx`, `components/digital-wallet-buttons.tsx` and the tutorial registry. Do not change the surrounding phone, tablet or desktop layout while doing it.

Check

- Every T1 wallet and simulation test passes.
- For each disabled route: correct status code *and* zero row changes, zero provider transport, zero success SSE, push, event, usage increment or notification outbox entry.
- Retried and concurrent disabled requests remain side-effect free.
- `APP_ENV=production` with `PAYMENT_MODE=simulation` or `uat` refuses to start.
- Device smoke at all three viewports — no layout shift from the hidden controls.

R0-T6Agent

Stop startup database dumps and the side-effectful build

Depends on: T1, T2

Startup dumps — `server/index.ts` ~216–232

The block spawns `scripts/db-backup.sh` on every boot and then daily, guarded only by `NODE_ENV !== 'production'`. Its own comment states it snapshots the workspace database *and* production Neon. Remove it entirely.

A development process must never read a production backup URL merely because the variable is present. Replace with an explicit operator-only command requiring a named target, typed confirmation outside CI, an encrypted destination and no checked-in artifact.

Side-effectful build — `.replit`

Deployment build is currently `["bash", "-c", "npm run build && npm run db:migrate"]`, so building connects to a database and applies migrations. Reduce to build alone; migrations move to a separately approved release step.

```
build = ["npm", "run", "build"]
```

Check

- The T1 backup test passes — starting in development, test or production spawns nothing.
- A source guard asserts no production database URL variable is referenced by normal development startup.
- `npm run build` completes with no database connection.
- `db-backups/` is gitignored and no dump is tracked.

Phase C — Secrets

T7 is the last agent task. Everything after it is yours.

R0.C

R0-T7AgentMust precede H2

Scrub the tracked configuration

Depends on: T5, T6 · Blocks: H2

Remove the entire secret block from tracked `.replit` — lines 72–77 hold `JWT_SECRET`, `ADMIN_EMAIL`, `ADMIN_PASSWORD_HASH`, `VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY` and `VITE_VAPID_PUBLIC_KEY`. Keep only non-secret run and deployment structure.

Add a key-name-only `.env.example`. Add ignore rules for local env files, database dumps, runtime uploads, trace archives and provider fixtures containing live data — keeping sanitised test fixtures tracked. Add a CI secret scan with redacted output and a reviewed allowlist. Write the rotation runbook covering JWT, admin, VAPID re-subscription and Windcave credentials.

Check

The T1 tracked-config test passes. A tree scan reports no unapproved secret. The app still boots locally reading from an ignored env file. Nothing is rotated yet — that is H2.

R0-H1You

Declare and inventory

Depends on: nothing — any time before H2

Record: incident owner, the window the secrets were exposed, repository visibility over that window, which revisions were deployed, and who or what had access. Short — but without it you cannot later answer "was anything exploited?"

Check

A dated record exists under `docs/evidence/remediation-v2-2/r0/`. No secret values in it.

R0-H2YouOrder-critical

Generate new secrets straight into Replit Secrets

Depends on: T3 (VAPID recovery), T7 (tracked file scrubbed)

Generate *new* values — do not copy the old ones across. Set them in Replit Secrets only; they must never re-enter a tracked file.

```
# JWT secret
node -e "console.log(require('crypto').randomBytes(48).toString('base64'))"

# payment return state — must be independent of JWT SECRET
node -e "console.log(require('crypto').randomBytes(48).toString('base64'))"

# admin password hash — choose a strong password first
node -e "require('bcrypt').hash(process.argv[1],10).then(console.log) "
'<new-password>'

# VAPID keypair
```

```
npx web-push generate-vapid-keys
```

Accept that rotating `JWT_SECRET` invalidates every existing session. Do not keep the old key as a grace verifier — a burned key with a grace period is still a burned key.

Check

All values set in Replit Secrets. None appear in `git diff`, test output, CI artifacts or evidence.

R0-H3You

Deploy and verify the rotation took

Depends on: H2

- A token signed with the old `JWT_SECRET` is rejected.
- The old admin password fails; the new one works.
- A newly authenticated session works end to end.
- Push: an existing subscriber re-subscribes cleanly (what T3 bought you) and receives a test notification.
- The app boots with no secret in tracked config.

Check

All five verified and recorded. If the old JWT still works, stop — that is a stop condition and means the deploy did not pick up the new value.

R0-H4You

Review access logs and scan history

Depends on: H1

Look for forged admin use, unusual logins, cross-merchant access, calls to `clear-transactions`, simulator calls, refund creation and API-key route access. With no live merchants the expected finding is nothing — but "we looked and found nothing" is materially different from "we assumed nothing happened".

Run a redacting secret scan over tree *and* history. Record rule ID, file, commit and status — never the matched value. Then decide separately whether history rewriting is warranted; rotation is mandatory either way, and rewriting is its own destructive operation with its own approval.

Check

Log review recorded. Scan output recorded with values redacted. History-rewrite decision recorded either way, with reasoning.

R0-H5You

Classify tracked uploads and local dumps

Depends on: H1

Inspect tracked files under `uploads/` — including `uploads/invoices/*.png` — with whoever owns privacy. If any is real customer data, move the authoritative copy to approved storage, remove it in a forward commit, and include the path in the history and retention review.

Separately inventory local `db-backups/*.sql.gz`. They are gitignored but may contain production data. Move or securely delete only with explicit approval and a retention record.

Check

Every tracked upload classified. Local dumps inventoried and dispositioned with a recorded decision.

R0.D

- Secret rotation and exposure review recorded by the incident owner (H1–H5).
- Fake-approval routes unreachable in a production-mode test (T5).
- Transaction clearing cannot delete, and `clearTransactions` is gone from all three storage locations (T4).
- Production seeding impossible under every flag value (T4).
- Automatic production dumps removed from startup; build makes no database connection (T6).
- All eight T1 tests pass, and the original failure output is preserved as evidence.
- `npm run check test:client test:server build` and the device smoke matrix pass.
- Existing legitimate provider reconciliation preserved or explicitly covered by the in-flight runbook.

Rollback constraint

Ro rollback is a code rollback *only* if it restores no fake-payment, seed, destructive or compromised-secret behaviour. If an Ro deployment is bad, keep payment initiation disabled while correcting forward.

Part 4 · After R0

Twelve tasks. Build the tests that prove runtime behaviour, establish the accepted visual baseline, and fix high-confidence correctness issues before touching payments.

R1.A

R1-T1AgentBlocks all of R1

No-live-system HTTP test harness

Depends on: R0 complete

Refactor route registration just enough to construct an Express app without listening, Vite, migrations, seeding, cron startup, backups or real provider clients. Inject or mock storage, Windecave transport, clock, push, SSE, outbound HTTP and configuration.

The existing 23 tests in `server/__tests__` are unit-level. This is the first runtime HTTP harness — static route-inventory tests remain useful but cannot substitute for it.

Create these principals as reusable fixtures:

```
merchantAOwner · merchantAMember · merchantBOwner  
disabledUser · validatedAdmin  
publicPaymentBearer · providerNotification · cronCaller
```

```
ecommerceApiKey // disabled by default
```

Check

A test can construct the app, issue an authenticated request as merchant A, and assert a response — with no network, no database, no cron and no Vite. Runs in under 5 seconds.

R1-T2Agent

Checked-in route policy inventory

Depends on: T1

Create `server/route-policy.ts` plus a generated, tested table in documentation. There are 215 route registrations to inventory. Each records:

```
method · path · principal type · roles
tenant source and ownership rule
strict path / query / body schemas
capability or payment-mode gate
billing / entitlement gate
idempotency scope
storage methods called
external side effects
success DTO
```

```
error disclosure policy
```

Add a test comparing registered routes against the inventory that fails when a route is added without policy. Inventory all 215; do not attempt to rewrite them all in one commit.

Check

All 215 routes appear in the inventory. Adding a dummy route without a policy entry fails the test. `npm run check` passes.

R1-T3Agent

Explicit role and tenant matrix

Depends on: T2

Record and test an owner / member / admin / public / provider / cron / API-key matrix. Safe defaults:

- `POST /api/merchant` owner and explicitly allowed admin only. Members cannot initiate by default.
- `POST /api/merchant` owner only unless product explicitly grants members.
- `POST /api/merchant` move to `/api/account/password` or prove the path merchant equals the authenticated account. Never ignore a path ID.
- `POST /api/merchant` requires the existing validated admin principal, not a user-controlled role claim alone.
- `POST /api/merchant` authorise one payment resource only and never grant merchant API access.

Check

Matrix recorded in the inventory. Each row has a runtime test proving allowed principals succeed and disallowed ones get the P2.2 status code with zero side effects.

R1.B

R1-T4AgentHighest security value

OAuth rebuild — and the reference implementation for three flows

Depends on: T1

The existing Google flow at `routes.ts:336-351` builds its authorize URL with `client_id`, `redirect_uri`, `response_type`, `scope`, `access_type` and `prompt` — and no `state`. The callback therefore has nothing to verify the response against, which is exactly the CSRF protection `state` exists to provide. Worse, on success at `:433-440` it does:

```
const token = generateToken(authUser);  
const redirectParams = new URLSearchParams({ token, merchantId: ... });  
  
return res.redirect(`/login?${redirectParams.toString()}`);
```

The bearer JWT goes in a URL query string — landing in browser history, referrer headers, and any proxy or analytics log along the way.

Required:

- Require `PUBLIC_ORIGIN` stop constructing security-sensitive URLs from untrusted `Host` or forwarded headers.
- Configure the exact trusted proxy depth for Replit. Test spoofed `X-Forwarded-For` host and protocol.
- One-time `state` `nonce` and PKCE. Validate issuer, audience and nonce.
- Exchange the callback for a short-lived one-time application code in an `HttpOnly/SameSite` flow.
- Move login and reset throttling from process memory to a shared store before multi-instance production.
- Apply canonical origin rules to password reset, invitations, checkout links, invoice links and provider callbacks.

Check

- Tests: replayed callback, wrong/missing state, wrong nonce/audience/issuer, PKCE mismatch, open redirect, spoofed host and proxy.
- A test asserts no JWT appears in any redirect URL or log line.
- Two-instance rate-limit sharing verified.

R1-T5AgentApple 4.8

Sign in with Apple — built on T4's corrected pattern

Depends on: T4

App Store Guideline 4.8 requires that an app offering third-party social login also offer an equivalent service limiting data collection to name and email and allowing the user to keep their email private. Google sign-in is live and reachable from inside the iOS app — `app-login.tsx` is a splash screen whose only action routes to `/login`. None of the guideline's exceptions apply.

Build it here, not as a separate workstream: T4 has just made the OAuth pattern correct, and building Apple sign-in before that means building against a route T4 replaces.

- Client: `@capacitor-community/apple-sign-in`, or a small native plugin calling `ASAuthorizationAppleIDProvider`
- Server: new `/api/auth/apple` and callback mirroring the corrected Google handler — verify Apple's identity token server-side, create or link a merchant record the same way.
- `state` nonce and PKCE. Do not copy the old parameter list.
- UI: same visual weight as the existing Google button, per Apple's Human Interface Guidelines for the mark.

Check

Sign in with Apple completes end to end on a real device. Identity token verified server-side. Same state/nonce/PKCE test suite as T4 passes for this flow. No JWT in any URL.

R1-T6Agent

Strict numeric parameter parsing — 71 sites in batches

Depends on: T2

One helper or middleware accepting only `/^[1-9][0-9]*$/`, converting with `Number`, verifying `Number.isSafeInteger`. A separate UUID parser for UUID paths.

Reject with 400: 0, negatives, plus signs, decimals, exponent notation, whitespace, suffixes like `1abc`, arrays or repeated query values, overflow and unsafe integers, missing or empty values.

Migrate in these batches — counts are route registrations per family on this baseline. Run that family's runtime tenant tests after each batch.

Batch	Family	Routes
1	/api/merchants	39
2	/api/transactions + /api/payments + /api/checkout + /api/pay	36
3	/api/trades	31
4	/api/admin	28
5	/api/property	21
6	/api/push + /api/auth + /api/team + /api/billing	32
7	remainder — subscription, windcave, tutorial, v1, internal, webhooks	20

Add a source guard forbidding new `parseInt(req.params)` usage. Do not blind-replace UUID paths or intentionally free-form values.

Check

After each batch: that family's tests pass and the permissive-parse count drops. At completion: zero `parseInt(req.params)` in `routes.ts`, source guard active, all 71 sites migrated.

R1-T7AgentBiggest tenant gap

Tenant-scoped storage — close the 81-route gap

Depends on: T2, T3, T6

Verified on this baseline: `authenticateToken` appears 116 times in `routes.ts`, but `checkMerchantOwnership` only 35. Some of that gap is legitimate — account-level and admin routes do not need merchant scope — but it is the highest-value audit in R1, and Ro's `clear-transactions` finding came from exactly this gap.

Audit every one of the 116 and classify: needs tenant scope, legitimately account-level, or legitimately admin. Record the classification in the route policy inventory.

Then prefer storage methods that *require* tenant scope over global reads followed by route-level comparison:

```
// prefer this
getTransactionForMerchant(transactionId, merchantId)

// over this — the shape that produced the clear-transactions bug

getTransaction(transactionId) // then compare in the route, or forget to
```

Implement for transactions, refunds, boards, stock, property clients and invoices, trades clients/quotes/invoices, settings, uploads and exports. Remember: three edits per method (P2 convention 1).

Check

- All 116 authenticated routes classified in the inventory.
- Every route needing tenant scope has a two-merchant runtime test proving merchant A cannot read, mutate, export, upload for, refund or delete merchant B data — and that the attempt has zero side effects.
- `npm run check` passes with both storage implementations updated.

R1.C

R1-T8Agent

Fix the hook-order crash

Depends on: R0 complete

Audit early returns that occur before later hooks in these six files — all verified present on this baseline:

```
client/src/pages/settings.tsx
client/src/pages/exports.tsx
client/src/pages/merchant-terminal.tsx
client/src/pages/merchant-terminal-mobile.tsx
client/src/pages/transactions.tsx
client/src/pages/stock-management.tsx
```

```
client/src/pages/payment-stack.tsx
```

Move authentication and loading guards above the page through route wrappers, or invoke hooks unconditionally and guard their effects. Do not disable the hook lint rule.

Convert `client/src/__tests__/zz-review-hooks-repro.test.tsx` from "expects crash" to "does not throw". Test loading→authenticated, loading→unauthenticated, error→retry, merchant change and unmount. Make tests fail on unexpected React console errors and repair the `act(...)` warnings recorded in Ro-To.

Check

The repro test asserts no throw and passes. No `act(...)` warnings remain. Hook lint rule is an error and passes on all seven files.

R1-T9Agent

Truthful frontend failure states

Depends on: T8

An essential query failure must never render as `$0`, an empty array, "no sales", "no saved card", or an enabled money CTA. Add in-frame `role="alert"`, a retry, and disabled actions for failures in: desktop property analytics and terminal; desktop retail analytics, stock and terminal; desktop trades analytics and terminal; desktop settings.

Optional-source failures show "partial data" or "unavailable", never a fabricated numeric zero. Mutations stay disabled while pending, preserve user input on failure, and prevent double submit. Billing 402 errors state the required action without conflicting duplicate banners.

Check

With each essential query forced to fail: no zero or empty-state rendering, an alert is present, money CTAs are disabled, and retry works. No duplicate banners on 402.

R1-H1You

Accept the device baseline commit

Depends on: T9 · Blocks: T10

"Mobile unchanged" is meaningless without a reference. Accept an exact commit as the visual baseline — from that point "unchanged" means unchanged from it, not from an obsolete screenshot.

Also decide D10: is a fine-pointer 1440×650 viewport a phone or a desktop? The current shortest-side algorithm classifies it as mobile. Do not let anyone silently change device gating without this decision.

Check

Baseline commit SHA recorded. D10 decided and written as an ADR.

R1-T10Agent

Device and tutorial acceptance matrix

Depends on: R1-H1

Context	Viewport / capability	Expected shell
Phone	390×844, coarse/touch	Current mobile UI only
Tablet	1194×834, coarse/touch	Shared tablet/desktop UI, full bleed
Desktop	1440×900, fine pointer	Centred 1180×880 rounded 13-inch frame
D10 case	1440×650, fine pointer	Per the R1-H1 decision

Also test public, auth and onboarding routes outside the merchant frame, and iPhone orientations chosen by product. Existing scripts: `npm run verify:mobile`, `npm run verify:terminal-dock`.

Add a real tutorial matrix at all three contexts: complete and restart each tutorial; verify visible target and fallback, resize and route-transition geometry, persistent progress, and no disabled Tap or NFC step (those are off per Ro-T5).

Check

Zero page errors, console errors, failed required requests, clipped critical text, missed tap centres, or terminal layouts with zero visible rows. Existing baseline allowances are characterisation only and must reach zero. Golden image updates require product approval.

R1.D

- Runtime harness and policy inventory cover all 215 registered routes.
- Cross-tenant tests use at least two merchants and prove no side effects; all 116 authenticated routes classified.
- Zero permissive numeric parsing remains; CI prevents new instances.
- OAuth passes state/nonce/PKCE tests and exposes no JWT in URLs; Sign in with Apple works end to end.
- The hook crash is fixed rather than asserted; no unexpected React console output.
- Essential frontend failures are truthful and non-actionable.
- Phone, tablet, desktop and tutorial evidence approved against the R1-H1 baseline.

Part 5 · Scoped now, decomposed when reached

Full scope, rules and exit gates. Not numbered tasks — because each phase's tasks depend on what its predecessor actually built, and inventing that now produces confident instructions that turn out wrong.

R2

Make provider configuration and responses typed, environment-pinned, sanitised and independently testable before enabling UAT initiation.

- `server/windcave.ts` receives validated config, clock, ID generator and HTTP transport. Tests use sanitised official-shape fixtures, never live network.
- `confirmed approved · confirmed declined/cancelled · pending/non-final · transport timeout/unknown · malformed/unparseable · integrity mismatch · configuration disabled`. Never convert timeout, malformed, missing field, ambiguous multiple transactions, or pending into "approved" or "declined".
- `URL` HTTPS only; exact host for the selected mode; port 443/default only; no userinfo; no query or fragment; exact approved API path prefix. UAT rejects production hosts and production rejects UAT hosts. Typed validator per link relation — API/query, Hosted Payment Page, Hosted Fields/AJAX, 3-D Secure, callback back to exact `PUBLIC_ORIGIN`. Validate every link before returning it to a browser or following it with credentials. Reject unknown relations, redirects to unapproved hosts, protocol-relative URLs, encoded host tricks, nonstandard ports, DNS-suffix lookalikes and mode mismatch.
- Shared parser converting canonical decimal strings to integer cents. Reject more than two fraction digits, exponent notation, signs where not permitted, non-NZD for current flows, and overflow. No floating-point money math anywhere.
- — expected amount cents, currency, merchant reference, provider environment/account marker, provider transaction ID, sanitised reconciliation codes. Immutable once the provider session is attached. The merchant reference uses a stable non-secret attempt identifier and domain prefix — never `Date.now()` as business identity. Add unique indexes on non-null processor session ID, X-ID and provider transaction ID after duplicate preflight.
- `verifyWindcaveOutcome(expected, observed)` returns a typed verified outcome or typed rejection. Approval requires exact match of: persisted provider mode/account, session ID, one unambiguous final transaction, operation type, integer cents, currency, merchant reference, expected local attempt/aggregate, provider authorisation state. On mismatch: do not finalise, keep reconcilable/blocked, record a sanitised security event and metric, return a processing state to the browser rather than exposing provider internals.

What already exists to extend

Migration 0011 and current services already provide hashed retail payment bearer tokens, durable `payment_attempts`, a single live attempt per retail transaction/share, session and X-ID binding, return-state hashing and expiry, row locking, leases, compare-and-set finalisation, and atomic transaction/split/counter updates. `server/payment-attempt-service.ts` is real. R2 this; it does not rebuild it.

R2 exit gate

No provider URL or query result bypasses strict parsing and allowlisting · all enabled completion paths call the shared verifier · no raw provider body, session or token is logged · sanitised UAT fixtures pass with no live call · existing retail durable-attempt tests stay green.

R3

Make callback and FPRN processing crash-safe and idempotent, and give every retained payment domain the guarantees the retail token flow already has.

Keep `payment_attempts` `server/payment-attempt-service.ts` its row-lock order, finite lease, live-attempt unique constraint, return-state hash and atomic finaliser. Extend additively with R2's expected provider fields.

Do force property/trades invoice IDs into the retail `transaction_id` foreign key, and do not weaken that key into an unvalidated polymorphic integer. Before the R3 migration, write a schema ADR choosing one of:

1. concrete `invoice_payment_attempts` with exactly one checked property-invoice or trades-invoice foreign key; or
2. separate property and trades attempt tables behind one shared service contract.

The ADR must prove referential integrity, tenant scope, share identity, token migration and a shared finalisation API. A generic `aggregate_type + aggregate_id` table with no database foreign key is rejected unless a reviewer accepts a documented enforcement mechanism.

Additive table plus worker, only after a restored-snapshot migration rehearsal. One logical row per provider + notification kind + provider session; persisted session identity and linked attempt; first/last receipt time and arrival count; `pending · processing · retry · complete · dead_letter` finite lease owner and expiry; attempt count and next-attempt time; sanitised last error code only; observed provider transaction ID after verified query; unique provider/session/kind index.

```
// FPRN handler – order is the whole point
1. validate method, content type, size, field types, ID length/shape
2. do NOT authenticate the notification by trusting its body – it is a hint
3. resolve NO money outcome from the notification fields
4. upsert the durable inbox row and COMMIT
5. return provider-appropriate 200 only after durable receipt
   (DB write failed → retryable non-2xx; never acknowledge and lose the event)
```

```
// worker
1. claim due rows with FOR UPDATE SKIP LOCKED + finite lease
2. query Windcave through the R2 transport
3. resolve the exact persisted attempt BY UNIQUE SESSION
   (never "next pending share")
4. run verifyWindcaveOutcome
5. pending / transport unknown → bounded exponential backoff with jitter
6. integrity mismatch → dead-letter + security alert, NO money mutation
7. final verified outcome → call the ONE aggregate finaliser

8. mark inbox complete only after the finaliser commits
```

The browser callback validates hashed return state and triggers the same reconciliation path. It must not contain a second money finaliser. Invalid or expired return state cannot change money; FPRN and the worker may still complete the legitimate persisted attempt. Terminal replays return the same outcome and do not re-increment usage, paid share count, GST, notification, push or SSE.

Typed transition modules for: retail transactions (at least `pending · processing · completed · failed · cancelled · partially_refunded · refunded`) · retail split shares · payment attempts · property invoice and split share · trades quote/invoice and split share · refunds · subscription card and billing attempts.

Public and domain statuses are not provider outcomes. Each transition lists permitted source states, actor, side effects, idempotent replay result and terminality. Routes cannot call generic `updateTransactionStatus(id, string)` in normal operation. Storage methods use compare-and-set and return claimed / terminal / conflict / not-found. Events append in the same transaction as the state change.

Messages publish from a durable outbox keyed uniquely to the event. Add CHECK constraints only after a count-only dirty-status preflight. No global state enum; no silent renaming of `completed` `failed` to `approved` `declined`.

Build a call-site and traffic inventory before deleting any route. Per flow choose exactly one: (404 tombstone then delete) · (session creation and completion move to a durable attempt plus shared verifier) · (no new sessions; existing provider identities remain queryable until their maximum lifetime expires).

Property/trades public link tokens are currently raw in storage. Migrate with expand/compatibility: add hash columns → write hash for new tokens while returning raw once → backfill hashes → keep emailed links working through temporary dual lookup → verify → remove raw-token reads in a later contract release. Never log either form.

R3 exit gate

Every retained payment flow uses a durable attempt and the shared verifier/finaliser, or is demonstrably unreachable · no callback or notification contains independent read-then-write completion logic · fault-injection suite passes: duplicate FPRNs before/during/after callback, two workers and two instances competing, crash after inbox commit / after provider query / after finaliser claim / after DB finalisation / before message publication, pending→approved and pending→declined, amount/currency/reference/session/share mismatches, lost browser return, expired and replayed return state, multiple provider transactions, split A cannot complete split B, counters increment exactly once, initiation flags off while reconciliation completes a pre-existing session.

R4

A retry or ambiguous network failure must never refund twice, release money prematurely, or claim a simulated success. `FEATURE_REFUND_INITIATION` stays false through this release.

- owner-only initiation by default; admin path needs explicit policy and an audit reason. Split refunds stay disabled until each refunded amount maps to a specific captured share and provider transaction. `original_payment_method`, manual/cash and bank transfer are distinct operations with distinct UI and audit language — a manual record must never update fields implying Windcave moved money.

- (forward migrations; drop no legacy columns): UUID public/operation ID · client idempotency key and request fingerprint with merchant-scoped uniqueness · stable provider X-ID created *before* the call and unique · original capture provider transaction ID and optional share identity · requested amount cents and currency · typed state (`requested` · `submitted` · `unknown` · `succeeded` · `failed` · `cancelled`) · provider refund transaction ID unique when non-null · claim lease, attempt count, next reconciliation time · sanitised last error code · initiator principal and reason code · timestamps. A repeated key with a different fingerprint is 409; the same key returns the same operation.
- lock the merchant-scoped transaction and share → verify owner permission and refundable state → compute remaining cents from confirmed successes plus live reservations → reserve → insert/reuse the operation with stable X-ID → append audit event →
 . Never reserve in one transaction and create the row in another. Do not update `total_refunded` or set status `refunded` until provider-confirmed.
- submit the persisted X-ID every time. A synchronous success still passes a strict verifier before local success. Confirmed decline transitions to `failed` and releases the reservation atomically. Timeout, reset, malformed response, unexpected 2xx or provider pending → `unknown` reservation retained. Only a confirmed "not processed" releases it. Never treat all 2xx as success; never generate the X-ID inside the provider call; never put raw provider error bodies in merchant responses or logs.
- allowlisted list and detail. No raw refund rows or internal provider IDs unless a reviewed owner contract requires a redacted reference.

R4 exit gate

Reconciliation worker and ambiguous-response tests pass: two concurrent partials at and over remaining balance · same key/same payload · same key/different payload · timeout before send · timeout after provider accepted · crash at every boundary · provider pending then success and then decline · duplicate provider refund ID · missing original capture ID · share mapping and cross-share rejection · member and other-tenant denial · one-cent and rounding edges · disabled flag has zero reservation, row, provider or message side effects · manual refund language never implies provider success. A happy-path UAT refund alone is insufficient.

R5

Remove plaintext at rest without corrupting live rows or losing rollback readability. Scope: the four legacy bank fields (name, branch, account holder, account number), plus a counted decision on merchant `windcaveApiKey` values — secure deletion if unused, or temporary encryption under this system until an approved deletion date. Do not encrypt historical Stripe identifiers; they are not credentials.

```
enc:v1:<key-id>:<base64url(iv)>:<base64url(tag)>:<base64url(ciphertext)>
```

```
AES-256-GCM · exactly 32-byte keys · random 12-byte IV per value · 16-byte tag
```

```
AAD binds schema/table + row ID + column name
```

```
explicit active key ID + keyring containing every readable key
```

```
no key material in DB, repo, log, error, dump or client bundle
```

```
// strict parser
```

```
enc:v1 malformed / unknown key ID / failed auth → ERROR + metric, never  
plaintext
```

```
unknown enc:vN → unsupported-version error
```

```
no prefix → legacy plaintext, only while  
the
```

```
compatibility reader is  
enabled
```

```
null → null
```

reader expansion — deploy keyring validation and a reader handling plaintext plus valid ciphertext; writes unchanged; exercise old-code/new-code compatibility on a restored snapshot. encrypted writer — write only `enc:v1` because AAD includes the row ID, insert the row first without sensitive values or use one transaction that obtains the ID then writes ciphertext before commit; reads stay dual-format. dry run then batched backfill — dry run outputs counts only (null, plaintext, active key, old key, unknown key, malformed, auth failure); stop on unknown/malformed/auth failure rather than overwriting; bounded batches with durable checkpoint and `FOR UPDATE SKIP LOCKED` encrypt from the locked old value and update with compare-and-set so a concurrent write is not lost; idempotent and resumable. verify and contract — recount to zero plaintext, sample-decrypt in an authorised isolated process comparing in memory without printing, add an envelope-shape constraint only after clean verification, disable plaintext reads in a later release once backup retention and rollback windows close.

Rollback rule

Once encrypted writes begin, rollback goes *only* to an E1-compatible reader. Never resume plaintext writes; never deploy a pre-dual-read build after ciphertext exists. If E2 or E3 has a problem, stop the writer and fix forward.

add the new key to readers → set active for writes → re-encrypt old-key rows in batches with the same CAS protections → verify zero rows need the old key → retain the old key until every retained backup containing its ciphertext expires or is re-encrypted. Perform a restore-and-decrypt drill before removing a key. Existing plaintext backups do not become safe because the live database was backfilled.

R5 exit gate

Zero plaintext in the live target · reviewed dry-run and backfill report · restore-and-decrypt proof · no new collection path · recorded retention decision and date. Tests cover round-trip per field/row AAD, wrong row/field AAD, tampered IV/tag/ciphertext, malformed envelope, unknown key, rotation, null/Unicode/max length, concurrent backfill and write, restart and checkpoint, idempotent rerun, partial batch failure, restore with keyring. If D8 stays provisional, go-live remains blocked even though the technical work is complete.

R6

- Current code requires both `lastBillingDate` and `currentPeriodEnd` for active access, while migration 0014 grants active periods without inventing a billing date — an earlier production snapshot found 8/8 rows affected. Do not resolve this by falsifying a historical charge. Access is granted by a current explicit entitlement period with recorded provenance (`paid · migration_grant · complimentary · trial · dunning_grace`); `lastBillingDate` means a real confirmed bill only. Sequence: product signs D9 and the dunning policy → add provenance via forward migration → produce a read-only production report (merchant count by state and provenance, period validity, missing card token, due/overdue, "would be blocked before/after", no PII or token values) → rehearse on a restored snapshot → update `subscriptionHasPaidAccess` and every `requireBillingCard` consumer through one shared tested policy, not route by route →
- with synthetic data: PCI-appropriate hosted/tokenised capture with PAN and CVC never persisted or logged · token survives a billing-period roll and a restart · the same X-ID yields one charge · two workers/instances cannot double-charge · transport timeout becomes unknown/reconciling rather than an immediate second charge · confirmed decline enters bounded dunning · access and grace change exactly per D9 ·

replacement, removal and expired-token behaviour truthful · no live card or production endpoint.

- Replace process-local `cronRunning` with a PostgreSQL advisory lock or durable lease — two Replit instances must not both run a cycle. Configure and document one external scheduler URL, auth header, cadence, timeout, retry and alert owner. Persist a parent run and per-job start/end/status/count/duration/sanitised error. Job-level idempotency remains required; the lease is not a substitute. Resume abandoned leases safely after expiry. Alert when no successful run occurs within two intervals, reconciliation backlog ages out, a billing claim is stuck, or dead-letters rise. Tests: two concurrent triggers, crash and restart, stale lease takeover, repeated scheduler retry, one failing sub-job while others report correctly, missing/wrong secret, timing-safe comparison, fresh and stale alerts.
- `/healthz` liveness only — once booted, no database or provider dependency. `/readyz` checks boot complete, `SELECT 1` cached migration/schema fingerprint valid, required config valid, not draining; 503 when not ready. Scheduler freshness belongs in authenticated ops status unless the deployment owner explicitly wants traffic removed when jobs are stale. On graceful shutdown set readiness false before closing the listener, stop new jobs and initiations, drain bounded HTTP/worker/SSE work, release leases safely.
- Accept only a bounded safe inbound `X-Request-Id` or generate a UUID; return it and attach to request, provider, worker, audit and job context. One-line structured JSON with level, event, request/job ID, route template, status, duration and sanitised domain IDs, all passed through a central redactor. Prohibit raw notification and query bodies, Authorization, cookies, payment bearer tokens and hashes, return state, bank fields, API keys, card tokens, OAuth codes, full provider session and transaction IDs, and arbitrary upstream error objects. Hash or truncate correlation identities under a documented policy — logs must not become a second credential store. Tests supply sensitive material at multiple nesting levels, in arrays, error causes, query strings, headers and provider responses, and prove absence from captured logs and 5xx responses.

R6 exit gate

Production snapshot report proves no intended merchant is falsely blocked · stored-card UAT charge/retry/decline evidence approved · two-instance cron tests prove one executor, durable history and stale alerts · health, readiness and graceful shutdown pass · structured logs carry request IDs and redaction tests pass.

R7

device gating and the persistent desktop shell in `client/src/App.tsx` the centred scaled desktop frame and full-bleed tablet behaviour, the tutorial provider/registry with desktop targets and portal geometry, lazy-with-retry and the global chunk boundary, or terminal presentation boundaries. Client changes here are narrow: remove false capability, make errors truthful, use shared route constants, prevent duplicate actions.

Tap to Pay reality gate — verified state

`src/plugins/TapToPayPlugin.swift` is outside the iOS app target, has zero SDK integration — all nine Windcave references are comments and commented-out code — and returns `STUB_TOKEN <timestamp>` after a timed delay.

`ios/App/CapApp-SPM/Package.swift` has no Windcave dependency.

`window.TaptPay` is called from `merchant-terminal.tsx` and

`merchant-terminal-mobile.tsx`. No entitlement evidence exists. Proxy availability is not proof of native capability.

`FEATURE_TAP_TO_PAY=false` server-side and capability response false · no production source returns `STUB_TOKEN` or simulates approval · remove `window.TaptPay` in favour of a typed imported Capacitor plugin once one exists · hide Paywave and Tap controls and tutorial steps unless server feature, supported device/OS, native implementation, entitlement, SDK initialisation and UAT configuration all report ready · retire the legacy `/nfc` simulator and placeholder wallet components.

the exact Windcave iOS SDK artifact, version, licence, API contract, merchant configuration and support agreement · verified Apple Tap to Pay entitlement and supported device/OS matrix · SDK added through the approved package mechanism with reproducible dependency metadata · plugin implemented in the actual iOS app target, no commented pseudocode or guessed SDK calls · one `useTapToPay` controller owning capability, lifecycle, cancellation, background/foreground, single-flight submission and typed errors · server creates the durable attempt and stable provider identity before the phone begins a tap · native output never trusted as final approval — server query verification decides · `npx cap sync ios` reproducible `xcodebuild` and real supported-device UAT including cancel, decline, timeout, background, offline, retry and duplicate tap.

Audit `capacitor.config.ts` `Info.plist` the Xcode project and generated assets as one unit: `production.server.url` and navigation intentional · narrow `allowNavigation` — the current list includes broad `*.replit.app` and `*.repl.co` entries that should go unless a documented flow needs those exact hosts · Windcave navigation matches the selected mode and approved HPP/3DS hosts · no arbitrary external navigation or credential-bearing redirect · orientation product-approved and tested · generated web assets rebuilt and synced only from the approved source commit, staged as one coherent rollover.

R7 exit gate

Full acceptance matrix at 390×844 phone, 1194×834 tablet, 1440×900 desktop and the D10 case, across retail, property and trades home/terminal/analytics/settings, stock, transaction and refund read states, team/settings, exports, login/signup/onboarding, complete and restarted tutorials, and public payment/receipt/result states for pending, approved, declined, cancelled and error — with zero page or console errors, failed required requests, clipping, tap misses, duplicate mutations or background overflow. Every visible merchant control is backed by a real enabled API or truthfully disabled. No mock business data.

R8

- Pin Node 22 via `engines` and a version file. Add `.github/workflows/verify.yml` with least-privilege permissions and no production secrets. Jobs: `npm ci` with lockfile unchanged · typecheck · client and server tests · production build with a source assertion that build made no DB connection or migration · PostgreSQL service job using a uniquely named test database and `TEST_DATABASE_URL` running migration-from-zero and the existing verifiers, with a safety script rejecting non-test URLs · isolated app start, wait for `/readyz`, run device/tutorial/chunk/terminal browser verifiers, tear down · secret scan with redacted findings · dependency vulnerability report with explicit severity triage and no blind major upgrades · sanitised failure artifacts only. Add scripts `verify:desktop` `verify:tutorial` `verify:chunk` `verify:ci` rather than relying on undocumented manual commands.
- The current ESLint config inherits JavaScript base rules into TypeScript and produces thousands of mostly configuration-driven errors. Correct globs, ignores, TypeScript globals and parser, and base rule overrides → ensure `react-hooks/rules-of-hooks` is an error → classify existing findings and remove false positives without blanket disables → first enforce zero new errors on changed files → burn

down the reviewed baseline in scoped commits → make `npm run lint -- --max-warnings=0` a release gate only when the baseline is zero. Unexpected React console warnings and open handles remain test failures before then.

- only after import-graph and runtime proof, then targeted `npm uninstall`, lockfile diff review, and rerun of `npm ci`, tests and build. Verified unused in this baseline: `connect-pg-simple`, `express-session`, `memorystore`, `openid-client`, `passport`, `passport-google-oauth20`, `passport-local` — but remove only after the R1 OAuth refactor settles. Do not remove both `framer-motion` and `motion`; both have imports. Never delete or casually regenerate the lockfile. Stripe packages and crypto-payment product code are already absent — do not manufacture work to remove them again.
- Audit `SECURITY.md`, `APPLE_PAY_GOOGLE_PAY_COMPLIANCE.md`, `DEPLOYMENT.md`, `replit.md`, `ios-README.md`, `scripts/deploy.js`, the product brief, WooCommerce README/readme.txt/changelog, and customer-facing legal copy. Remove or qualify claims of "production ready", PCI/SAQ status, all-endpoints-protected, bank-level encryption, native wallet or Tap-to-Pay support, automatic monitoring or failover, working API keys, and automatic seeding — unless current evidence proves them. `ios-README.md` cites Guideline 3.1.5(a) for the point-of-sale exemption; that is the cryptocurrency section. The correct rule is `FEATURE_ECOMMERCE_API=false`. Product and legal approve published wording; do not edit legal promises as casual copy cleanup.
- Keep `FEATURE_ECOMMERCE_API=false`, remove hardcoded keys and merchant-1 behaviour, and mark the WooCommerce plugin unsupported until a separately approved API design exists. If later implemented it requires: cryptographically random raw key returned once · digest-only storage plus prefix and last-four metadata · merchant scope, typed permissions, expiry, revoke and rotation, distributed rate limit · no hardcoded merchant · signed outbound webhook outbox with protected secret · URL canonicalisation and SSRF protection · idempotency and durable payment attempts · end-to-end plugin UAT and truthful marketplace documentation. OpenAPI and full ecommerce work remain deferred.
- Obtain written provider, legal and accounting answers: who is merchant of record · which Windcave account receives each capture · is submerchant identity required per request · how are merchants onboarded and paid out · who owns processor fees, refunds, disputes, reserves and chargebacks · which reconciliation reports are authoritative · is TaptPay holding or transmitting client money. If TaptPay must settle merchants,

create a separate approved accounting design for an append-only balanced ledger, settlement batches, immutable source references, reversals, reconciliation and audit exports. Existing `merchant_settlements`, historical `platform_fees` or transaction events are not proof of a working ledger. This becomes a go-live prerequisite, not an improvised migration.

- Name the exact release SHA and lock hash · verify target identity and read-only migration status/fingerprint · create a managed encrypted snapshot and record its ID privately · restore to an isolated scratch Neon project with no external jobs, messages or provider credentials · verify representative row counts, constraints, indexes, FK actions, status distributions, tenant isolation and decryptability without printing data · record RPO/RTO and rollback timing · rehearse each forward migration with lock and statement timeouts · run full CI plus fault tests against the clone · deploy the expansion migration through the separately approved step, never through build · deploy code with all initiation flags false and `PAYMENT_MODE=disabled` · verify health, readiness, login, tenant isolation, reconciliation backlog, cron lock and freshness, logs and redaction, device smoke · enable UAT capabilities one at a time with synthetic merchants, rolling back a flag on any anomaly.

Part 6 · Config track starts now · submission after R3

Nine tasks. Five have no dependency on remediation at all and should start on day one, in parallel with Ro.

A.0

Item	Status	Where it happens
Wallet routes fake approval, unauthenticated	Blocking	R0-T5
Sign in with Apple (Guideline 4.8)	Blocking	R1-T5
In-app account deletion (5.1.1(v))	Blocking	A-T5 — starts now
Offline / unreachable-server state (2.1)	Blocking	A-T4 — starts now

Privacy manifest	Add & verify	A-T2 — starts now
Privacy policy exists, not linked in-app	One link	A-T6 — starts now
Info.plist entitlements, export compliance	Needs work	A-T1, A-T3 — start now
Tap to Pay: stub, no SDK, outside app target	Stays off	R7
IAP exemption cites the wrong guideline	Doc fix	R8
Capacitor allowNavigation breadth	Narrow	R7

Submission waits for R3 — and the demo account collides with R0

Until then the app's wallet payment path can report success with no provider call. That is a Guideline 2.1 rejection risk, but more importantly it would mean shipping a lie to real merchants.

Guideline 2.1(a) requires working reviewer credentials with real data behind them. Ro-T4 makes production demo seeding impossible. The naive resolution — re-enable the seed for App Review — reintroduces exactly the vulnerability Ro closes. Instead create a dedicated review merchant with a strong generated password, seeded with representative non-PII transactions, rotated after each review cycle. It is an operational account, never a code path.

A.1

A-T1Agent

Info.plist usage strings and export compliance

Depends on: nothing

ios/App/App/Info.plist is close to the Capacitor default — verified zero UsageDescription keys and no ITSApUsesNonExemptEncryption.

- Any native API called without its matching `NS...UsageDescription` is an automatic mechanical rejection. Audit against whichever native APIs are actually wired in and add the matching strings — `NFCReaderUsageDescription` for Core NFC, `NSCameraUsageDescription` if the camera is used natively, `NSFaceIDUsageDescription` for biometric unlock.
- Set `ITSAAppUsesNonExemptEncryption` TaptPay uses only standard HTTPS/TLS, so this is almost certainly `false` — setting it explicitly skips App Store Connect's manual encryption questionnaire on every build upload.
- `UIRequiredDeviceCapabilities` is currently `armv7` a 32-bit-era value inherited from the template. Harmless but stale for an arm64-only app; tidy while the file is open.

Check

Every native API in use has a usage string. A build upload no longer prompts the encryption questionnaire. App still launches on device.

A-T2Agent

Privacy manifest

Depends on: nothing

No `PrivacyInfo.xcprivacy` exists under `ios/`. Since 12 November 2024 Apple rejects uploads using a "required reason" API without declaring it — the `ITMS-91053` notice.

Honest scoping: enforcement triggers on required-reason API usage; Capacitor ships its own framework manifest from v5.7.4 onward and this project is on 8.3.0; and the only plugin here is `@capacitor/push-notifications`, not the `preferences` or `filesystem` plugins that most commonly pull in `UserDefaults`. It may not fire at all. That argues for verifying rather than assuming — not for skipping, since it is about an hour's work and must agree with the App Store Connect questionnaire regardless.

Create `ios/App/App/PrivacyInfo.xcprivacy` declaring `NSPrivacyTracking` and `NSPrivacyTrackingDomains` (check against the `@google-analytics/data` dependency), `NSPrivacyCollectedDataTypes` (financial and transaction data, contact info, device identifiers for push), and `NSPrivacyAccessedAPITypes` — if `UserDefaults` is in scope the reason code is `CA92.1`.

Check

File validates with `plutil -lint`. Push a TestFlight build and confirm Apple's automated scan flags nothing — one upload settles whether it was ever required.

A-T3Agent

Entitlements file

Depends on: nothing

No `.entitlements` file exists anywhere in `ios/`. `ios-README.md` documents adding "Near Field Communication Tag Reading" and "ProximityReader" under Signing & Capabilities — that step has not happened or has not been committed. Create and commit the file with only the capabilities actually in use. Do not add ProximityReader until R7's gate is met and Apple has approved the request.

Check

Entitlements file committed and referenced by the Xcode target. Build signs successfully. No capability present that the app cannot actually use.

A-T4AgentBlocking

Native offline / error screen

Depends on: nothing

`capacitor.config.ts` points `server.url` at `https://taptpay.co.nz/app-login`, so the bundled `webDir` assets are never used. If the site is slow, down, or the device is on a poor connection, the app is a blank white WebView — no message, no retry, no branding. App Review tests on constrained networks and "the app displayed a blank screen" is a routine Guideline 2.1 rejection. It is easy to miss precisely because it never happens on a developer's fast connection.

Give the WebView a native failure path: a bundled offline screen with TaptPay branding and a Retry button, shown when the load fails. In Capacitor that is a `WKNavigationDelegate` failure handler in `AppDelegate.swift`, or the `errorPath` option pointing at a bundled fallback page. It also materially strengthens the Guideline 4.2 argument that this is a real app rather than a wrapper.

Check

On a real device with networking disabled, launching the app shows the branded offline screen — not a blank view. Retry recovers once the network returns. Verified at all three viewports.

A-T5AgentBlocking

In-app account deletion

Depends on: A-H1 (retention decision)

No account-deletion route or UI exists. Guideline 5.1.1(v) requires deletion from inside the app, not only by emailing support, since TaptPay supports account creation.

This has real design questions behind it, which is why it is not a checkbox: a payment processor cannot hard-delete a merchant with settled transactions and GST records attached. The flow is account deletion plus clearly-stated retention of the financial records NZ law requires — said plainly at the point of deletion, rather than quietly doing something narrower than the button implies.

Remember P2 convention 1 — the storage methods need all three edits.

Check

A merchant can delete their account from Settings. Auth is revoked and login fails afterwards. Financial records are retained per the A-H1 decision, and the UI stated that before confirming. Deletion is idempotent and cannot be triggered for another merchant.

A-T6Agent

Link the privacy policy from inside the app

Depends on: nothing

The policy already exists — a 254-line NZ Privacy Act 2020 document in `client/src/pages/legal.tsx`, routed at both `/privacy` and `/terms` in `App.tsx`, with a named privacy officer. It satisfies the App Store Connect metadata half of 5.1.1(i) as it stands.

What is missing is small: 5.1.1(i) also wants the policy "easily accessible within the app". Today the only links come from the legal page's own footer and one line in merchant onboarding pointing at `/terms`, not `/privacy`. Nothing in the authenticated merchant app links to either. Add a link from Settings using that file's existing `Section` component pattern.

Check

A logged-in merchant can reach the privacy policy from Settings in one tap, at all three viewports. No layout regression.

A.2

A-H1You

Decide the account-deletion retention policy

Blocks: A-T5

What exactly is deleted, what is retained, and for how long? Financial records and GST evidence have statutory retention in NZ. Write it as an ADR and as the user-facing wording shown at the deletion confirmation.

Check

ADR recorded. User-facing wording approved. A-T5 can proceed.

A-H2You

Confirm the App Store Connect legal entity

Depends on: nothing

Guideline 5.1.1(ix): apps in highly regulated fields — payments explicitly among them — should be submitted by a legal entity providing the services, not an individual developer. Confirm the account is enrolled under TaptPay's registered NZ business entity rather than a personal Apple ID. Expect Apple may ask follow-up questions about payment facilitation and licensing; having that answer ready in the reviewer notes avoids a round trip.

Check

Account entity confirmed and recorded. If it is currently personal, the transfer is started — this can take time, so do it early.

A-H3YouAfter R3

Prepare submission materials and submit

Depends on: R3, A-T1...A-T6, R1-T5

- the manually provisioned review merchant described in A.0 — never a seed code path.
- show the app in use — terminal, a transaction, a receipt — not the splash or login screen.
- state specifically that this is a physical point-of-sale app for real-world card payments, exempt from in-app purchase under ; describe the native capabilities actually present (push notifications, and NFC only if R7 has completed); give a test path for completing a payment end to end. Generic descriptions get rejected.
- Business or Finance; a POS tool with no user-generated or mature content rates at the bottom of the age scale. Answer the questionnaire deliberately rather than leaving defaults.

Check

TestFlight build passes internal testing — push, offline state, Apple Pay/Google Pay buttons, Sign in with Apple all verified on device. Then submit. Apple review typically 24–48h.

Part 7 · Phase 0 starts now · build after R4

Twelve tasks. Phase 0 is pure admin with no code dependency and should start immediately; the build waits for R4.

x.0

A standard integration is free, with a that lifts after Xero's certification review. Certification is a covering API risk rating, authentication, encryption, personnel security and audit logging — so R5's encryption and R6's audit logging are the substance of what gets assessed. The only paid option, Custom Connections, is single-organisation and irrelevant here.

One thing to confirm with Xero in Phase 0

Several third-party sources (Truto, Accounting Today, Databuzz, Codat) describe a tiered usage-based model with a \$0 tier capped at 5 connections and paid tiers billed

on data egress, effective 2 March 2026. That does not match Xero's own OAuth2 documentation, which states the free/25-connection position. The discrepancy may be a proposal that changed before shipping. Xero's docs are the baseline this plan follows — but since 25 versus 5 would materially change the rollout, ask Xero developer support directly.

Decision	Settled as	Why it is not open
When to push an invoice	On payment only	Pushing on send requires creating <code>AUTHORISED</code> (non-editable) invoices in a live ledger. v1 only ever creates already-settled invoices. Keep the <code>xero_sync_trigger</code> column for later; hard-code the on-paid path
Which vertical first	Trades	Simpler one-off shape. Property reuses the same code once Trades is proven
Fee postings to Xero	Not in v1	Add the column, default false, write no code behind it
Multiple orgs per merchant	One only	If <code>/connections</code> returns more than one, show a picker and store a single row
Encryption	R5's envelope	A key-less format cannot rotate. One utility, one keyring
Worker concurrency	2 per tenant	Xero's cap is 5 concurrent per organisation; 2 leaves headroom for interactive calls

```

access token      30 minutes
refresh token     60 days if unused · SINGLE-USE, rotates every refresh
retry grace       30 minutes – a used refresh token still works in this
window,
                  so a failed refresh RETRIES rather than marking the
connection dead
headers           Authorization: Bearer ... + xero-tenant-id: ... on every call

rate limits (per organisation)  60/min · 5,000/day · 5 CONCURRENT

```

across all organisations 10,000/min app-wide

on 429 honour the Retry-After header

The refresh rule that corrupts connections if you get it wrong

Persist the new refresh token *before* using the new access token for anything. If the process dies between refreshing and saving, the connection is unrecoverable and the merchant must reconnect manually. The 30-minute grace window is the safety net — use it.

X.1

X-HIYou

Register the app and ask the two questions

Depends on: nothing · Answers take days — ask early

- Register a integration in the Xero developer portal (not a Custom Connection).
- Set redirect URIs per environment — they must match byte-for-byte including any trailing slash.
- Confirm the exact granular scope strings in the portal: `accounting.contacts`
`accounting.invoices` `accounting.payments` `accounting.banktransactions`
`accounting.settings.read` `offline_access` `openid profile email` No payroll, files, projects, assets or reporting scopes.
- (1) is the free Code-flow connection limit 25 or 5? (2) what does certification require in practice, and what is the lead time?

Check

App registered, client ID and secret stored in Replit Secrets (never a tracked file). Both questions asked, with answers recorded as an ADR when they arrive. Confirm a free Demo Company is available — that is where every build task is exercised.

X.2

Ordered. Each task's check must pass before the next begins. All work happens against Xero's Demo Company; no real merchant connects until the pilot.

#	Task	Dependencies	Acceptance check
X-T1	Schema — <code>xero_connections</code> , <code>xero_sync_log</code> and nullable columns on <code>clientProfiles</code> , <code>tenantProfiles</code> , <code>jobInvoices</code> , <code>invoicesRentRequests</code> , <code>refunds</code> , <code>merchants</code>	R4, R5	<code>npm run check</code> passes; dev restart auto-applies with no <code>--force</code> ; both tables exist; no existing table lost a column
X-T2	Storage layer — 8 methods added to <code>IStorage</code> , <code>MemStorage</code> stubs, <code>DatabaseStorage</code> implementations	X-T1	<code>npm run check</code> passes (tsc catches a missing <code>MemStorage</code> method); app boots with no <code>DATABASE_URL</code>
X-T3	OAuth2 connect and callback in <code>server/xero.ts</code> , routes inside <code>registerRoutes</code> , gated on <code>FEATURE_XERO_SYNC</code> . Follows <code>server/windcave.ts</code> structure and R1-T4's corrected OAuth pattern	X-T2 / R1-T4	Connect to the Demo Company end to end; a row appears; stored tokens are unreadable ciphertext; mismatched <code>state</code> is rejected; flag off → routes 404
X-T4	Token refresh — refresh when within 5 minutes of expiry; persist the new refresh token first; implement the 30-minute grace retry	X-T3	Set expiry to the past → next call refreshes and succeeds; both encrypted columns changed; an invalid refresh token sets <code>expired</code> rather than crashing

X - T 5	Settings UI — Integrations section using the existing <code>Section</code> component in <code>settings.tsx</code> ; official Xero-branded connect button; clearing/sales account pickers; disconnect; sync log	X-T3	A merchant with no connection sees a connect button and no errors; connect → disconnect → reconnect works; disconnect removes stored tokens; no other Settings section changed
X - T 6	Tax-rate cache — fetch once per connection via <code>accounting.settings.read</code> , cache in <code>taxRatesCache</code> refresh weekly	X-T4	Rates cached on connect; a second sync makes no settings call; stale cache refreshes after a week
X - T 7	Contact sync — <code>GET /Contacts?where=EmailAddress=="..."</code> before create; cache <code>ContactID</code> skip entirely when cached	X-T6	Sync one client → one Contact; sync twice → still one Contact and the second run makes no POST
X - T 8	Invoice sync, Trades only — <code>Type: ACCREC</code> <code>Status: AUTHORISED</code> <code>LineAmountTypes</code> mapped from <code>tradeGstMode</code> , <code>TaxType</code> from cache, cents→dollars, <code>Idempotency-Key</code> <code>UUID</code> , guarded by <code>xero_invoice_id</code>	X-T7	on an inclusive-GST <i>and</i> an exclusive-GST merchant; GST matches the emailed receipt exactly; re-running creates no duplicate; a forced Xero failure leaves the

			payment, receipt and email completely unaffected
X-T9	Payment allocation — into the merchant's chosen Clearing account, not their trading account	X-T8	Payment lands in the clearing account; the invoice shows paid; bank-feed reconciliation is possible without double-counting
X-T10	Retry worker <code>server/xero-sync-cron.ts</code> following <code>trades-cron.ts</code> — drain per tenant with honour <code>Retry-After</code>	X-T9	Queue 20 entries → never more than 2 concurrent per tenant; a 400 marks <code>failed</code> without retry; a 429 reschedules without incrementing <code>attemptCount</code> exhausted rows surface in the Settings sync log
X-T11	Credit notes — using <code>refunds.refundAmount</code> which supports partial refunds	X-T10	A partial refund produces a credit note for the partial amount, not the full invoice
X-T12	Property invoices — extend X-T8's code path to <code>invoicesRentRequests</code>	X-T11	A rent invoice syncs with correct GST; full Trades regression still passes

Two ways to silently corrupt a merchant's GST

1. `LineAmountTypes` `tradeGstMode` `Map "inclusive" → "Inclusive"` and `"exclusive" → "Exclusive"`. Getting this backwards does not error — it changes the invoice total by 15% and files wrong GST.
2. `TaptPay` stores cents everywhere (`amountCents` `unitPriceCents`); Xero expects decimal dollars. Convert with `cents / 100` and verify the returned Xero total equals the local total before marking a sync successful.

And never recompute GST for Xero — reuse the totals already computed by `shared/trades-gst.ts`. Two sources of truth for tax is a money bug.

The non-negotiable rule

Every sync operation runs `retry` the real TaptPay action has already succeeded — asynchronously, best-effort. A Xero timeout, expired token or 429 can only ever affect the Xero copy of a record, never the payment, the invoice being sent, or the receipt email. Failures land in `xero_sync_log` for retry and surface as a status badge in Settings, not as an error in the merchant's workflow.

- OAuth2 only — legacy Xero API keys are deprecated.
- Web app, not Custom Connection (single-tenant, wrong for multi-merchant SaaS).
- Granular scopes from registration; minimum necessary — no payroll, files, assets or reporting.
- Tokens encrypted at rest with AES-256-GCM — meets Xero's stated symmetric-encryption requirement and is directly assessed at certification.
- TLS 1.2+ in transport (already the case for outbound HTTPS).
- Audit logging via `xero_sync_log` plus the redacting logger pattern from `windcave.ts`
- Rate-limit respectful — no polling loops, bounded worker concurrency, `Retry-After` honoured.
- Official "Connect to Xero" button using Xero's supplied brand asset and blue — not a custom-styled button.
- Never described as "Xero-certified" or "Xero-endorsed" until the assessment has actually been passed.
- Disconnect deletes the stored connection and tokens; data already in the merchant's own org is theirs to manage.

- Privacy policy gains a section disclosing the new data flow to Xero — contact names and emails, invoice lines, GST numbers. An NZ Privacy Act 2020 obligation distinct from Xero's terms. The policy already exists at `legal.tsx` this is a new section, not a new document.

Part 8 · Applies to every phase

Evidence requirements, the release command gate, commit order, stop conditions, deferred work, review templates and the final go-live gate.

P8.1

Every row needs automated evidence unless marked human or provider.

Risk	Required proof
Committed secrets	Redacted tree and history scan, rotation record, old-key rejection
Cross-tenant access	Two-merchant dynamic HTTP tests for read, write, delete, export, upload, refund
Demo login	Production-mode startup and seed tests plus read-only target check
Fake payments	Mode matrix and zero-side-effect tests for every retired or disabled route
Route inputs	Strict numeric, UUID, query and body tests; source guard against permissive parse
Provider SSRF / domain	URL parser adversarial matrix and redirect tests
Provider authenticity	Query-session exact field match fixtures and mismatch alerts
Duplicate payment	Same key, different key, concurrency, callbacks and FPRN, crash and restart

Split payment	Share binding, exact cents, no cross-share completion, one counter increment
Refund	Stable X-ID, unknown reconciliation, balance lock, no premature release
Subscription	Entitlement snapshot report, idempotent UAT charge, dunning policy
Encryption	Tamper, AAD, rotation, CAS, backfill and restore tests; zero plaintext count
Scheduler	Two-instance lease, persisted history, stale alert, retry and restart
Logs	Nested sensitive-value redaction, request ID propagation
Hooks / frontend	No hook crash or console warning; loading, error and empty truthfulness
Devices / tutorial	Approved phone, tablet and desktop visual and interaction matrix
Native	Build plus real supported-device UAT; otherwise the feature stays off
Compliance	Named professional signoffs and corrected claims

Scripts are created by their own phase — do not assume a name exists before then.

Existing today: `check` `test` `test:client` `test:server` `test:server:postgres`,
`verify:mobile` `verify:terminal-dock` `build` `db:migrate:status`

```
npm ci
npm run check
npm run lint # once the lint baseline reaches zero (R8)
npm run test:client -- --runInBand
npm run test:server -- --runInBand
npm run test:server:postgres # isolated TEST DATABASE URL only
npm run build
npm run verify:chunk # added in R8
npm run verify:desktop # added in R8
npm run verify:mobile
npm run verify:terminal-dock
npm run verify:tutorial # added in R8

git diff --check
```

The CI orchestrator starts and stops its own isolated server. Browser verifiers on any unexpected console, page or request error — they do not merely print it.

P8.2

Numbering may shift; dependency order may not. Keep commits reviewable and do not mix unrelated churn.

#	Commit	Phase
1	security: add failing P0 route and startup tests	R0-T1
2	security: centralize fail-closed runtime configuration	R0-T2
3	security: retire transaction clearing and production demo seed	R0-T4
4	payments: quarantine simulation wallet NFC refund and mock API surfaces	R0-T5
5	security: scrub tracked configuration and add secret-scan guard	R0-T7
6	ops: remove automatic production dumps and side-effectful deploy build	R0-T6
7	test: add injectable HTTP app harness and route policy inventory	R1-T1/T2
8	security: add strict ID parsing and tenant-scoped route batches	R1-T6/T7
9	auth: harden OAuth origin proxy trust and shared throttling	R1-T4/T5
10	fix: repair hook order and truthful frontend failure states	R1-T8/T9
11	test: approve current device and tutorial baseline	R1-T10
12	payments: type Windcave transport URLs and query outcome verifier	R2

1 3	db: add expected provider identity and durable notification expansion	R3
1 4	payments: converge callbacks FPRN and invoice attempts	R3
1 5	payments: enforce aggregate state machines and one finalizer	R3
1 6	refunds: add durable idempotent refund and reconciliation workflow	R4
1 7	crypto: deploy legacy-field compatibility reader	R5 E1
1 8	crypto: enable encrypted writes	R5 E2
1 9	ops: add dry-run and resumable encryption backfill	R5 E3
2 0	billing: implement approved entitlement provenance	R6
2 1	ops: add durable cron leases health readiness and structured logs	R6
2 2	native: remove stubs and gate Tap to Pay capability	R7
2 3	ci: add full isolated verification pipeline	R8
2 4	chore: remove proven-dead dependencies and mock remnants	R8
2 5	docs: replace unsupported production and compliance claims	R8

Encryption commits 17–19 must not be squashed into a release that prevents compatibility rollback. Schema expansion and code deployment evidence stay distinguishable. The Apple and Xero tracks commit alongside these on their own branches.

Stop the phase, leave money initiation disabled, preserve evidence, escalate

- A secret appears in a diff, log, artifact, screenshot or response.
- Old rotated credentials still work unexpectedly.
- Target database identity is uncertain.
- Migration status shows pending, drifted, orphaned or unexpected applied entries.
- Backup, restore or decrypt proof fails.
- A migration exceeds its rehearsed lock or statement budget.
- Merchant A can read, mutate, export, upload for, refund or delete merchant B data.
- Production-mode code can reach simulation, UAT, a stub token or an unapproved provider host.
- Amount, currency, reference, session, share or provider environment mismatch can finalise.
- Duplicate, concurrent or lost-response testing can double-charge, double-refund, double-count or double-notify.
- An ambiguous refund releases its reservation.
- Subscription policy would block an intended active merchant.
- Scheduler lease permits two money-job executors, or required jobs are stale without alert.
- Plaintext backfill count rises unexpectedly, an envelope cannot decrypt, or an old-key backup cannot restore.
- A hook crash, page error, console error, false-empty business state, device regression or broken tutorial appears.
- Real funds-model, PCI, native, privacy or legal claims lack named professional approval.

Do not mark a phase complete because time or context is running out. Record it incomplete with the exact blocker. Do not relabel an unexplained failure "flaky".

Do not start these unless a gate promotes them into scope. Promotion requires an ADR and reviewer approval.

- Organisation hierarchy and generalised RBAC beyond the explicit route matrix; merchant-model normalisation; locations, terminals and device management.
- Full public ecommerce API, OpenAPI contract, generated typed client, pagination strategy, generalised reporting, and the WooCommerce marketplace release.
- Tap to Pay implementation before SDK and entitlement contracts exist.
- Crypto payments. Deletion of historical Stripe schema.
- A financial ledger unless D12 proves TaptPay owns settlement obligations.
- Broad load testing, disaster-recovery drills beyond release restore proof, fraud engine, payout tracking, synthetic monitoring, independent penetration test.
- Cosmetic redesign unrelated to truthful error and capability states.
- Legal or compliance conclusions reached by a coding model.

P8.5

Before any phase is implemented, the reviewer reads the proposed diff, ADR and preflight and returns exactly these sections:

```
## Verification of Prior Fixes
## Blocking Issues
## High-Risk Concerns
## Missing Steps
## Unsafe Assumptions
## Required Ordering Changes
## Open Product / Provider / Legal Questions
## Compliance and Data-Handling Notes
## Test and Rollback Adequacy

## Final Recommendation (Approve / Do not approve)
```

An *Approve* must cite evidence for every previously blocking issue and state the exact commit and phase scope approved. A second reviewer, or a separately recorded independent reread, checks the first review did not miss unsafe interactions. New blocking evidence returns the phase to review.

Phase / release:
Exact branch and commit:
Base SHA:
Scope completed:
Files changed:
Migrations (names / checksums / target; or none):
Preflight queries and count-only results:
Commands run and results:
Original failing-test output: (evidence the defect was real)
Negative / no-side-effect tests:
Provider / UAT activity (synthetic only, IDs redacted):
Device / browser evidence:
Security / privacy review:
External actions performed by whom:
Feature flags and payment mode after deploy:
In-flight operations / reconciliation status:
Known warnings or deferred items:
Rollback commit and rollback constraints:
Approvals:
Stop conditions checked:
Next phase prerequisites:

The next implementer starts from this handoff and verifies the recorded state. They must not infer completion from a green test count, a migration filename, a schema comment, a UI control, or an old "production ready" document.

P8.6

Production money initiation can be considered only when all applicable items are checked and evidence points to the exact final SHA.

- Incident closed; all burned credentials rotated; old credentials rejected.
- Tree and history secret findings reviewed; no unapproved secret remains.
- Demo seeding impossible; known demo login disabled in production.
- Destructive transaction clearing removed or tombstoned with zero-delete tests.
- All 215 routes in the policy inventory, passing runtime role and tenant tests.
- OAuth, canonical origin, proxy trust, rate limiting, strict IDs, file authorisation, DTOs and redaction all pass.

- Mobile appears only under the approved phone contract; tablet and desktop share the UI; desktop stays centred in its rounded 13-inch frame.
 - Every visible merchant control is backed by a real enabled API or truthfully disabled; no mock business data.
 - Tap to Pay remains off, or real SDK, entitlement, build and device-UAT evidence is approved.
 - Security, deployment, product, iOS, wallet and WooCommerce documentation matches reality.
 - PCI, AML/KYB, sanctions, chargeback, privacy and retention, merchant-of-record and settlement reviews completed by qualified owners.
 - Sandbox beta completed with synthetic merchants and fault injection before any production enablement.
-
- `FEATURE_XERO_SYNC` submitted after R3 with a manually provisioned review account; Sign in with Apple shipped in R1; approved and live.
 - `FEATURE_XERO_SYNC` pilot merchants connected against the Demo Company first; totals match to the cent on inclusive and exclusive GST merchants; D17 answers received; `FEATURE_XERO_SYNC` enabled only after its own evidence is approved.

Required approvers

Engineering owner, security owner, database owner, operations owner, payments/provider owner, product owner, and the relevant legal, compliance and accounting professionals.

P9

Every "required" item needs a named human. No coding model may resolve one by guessing.

I	State	Decision
D		
D 1	Locked	Treat all production data as live — rows, uploads, logs, history, backups

D 2	Locked	Retain historical Stripe columns; no drops in this remediation
D 3	Locked	Replit + Neon; design for multiple instances
D 4	Provisional	Platform-owned Windcave credentials — UAT only until D12
D 5	Locked	Crypto off via false-only <code>FEATURE_CRYPTON</code> invariant
D 6	Locked	Synthetic beta merchants on Windcave UAT first
D 7	Resolved	Branch from <code>origin/feat/tablet-desktop-app</code> @ <code>9a51bd8</code> — confirmed present. Reconfirm no other branch must merge first (Ro-To)
D 8	Provisional	Encrypt and retain four legacy bank fields; no new collection; legal owns retention
D 9	Required	Entitlement semantics for migration-granted merchants — needed before R6
D 10	Required	Is a fine-pointer 1440×650 viewport a phone or a desktop? Needed at R1-H1
D 11	Locked	Only owners and authorised admins initiate refunds
D 12	Required	Merchant of record, settlement, chargeback and refund ownership — before real money
D 13	Locked	Tap to Pay off until SDK, entitlement, build, server attempt and device UAT exist
D 14	Locked	Xero starts after R4; App Store submission after R3; Apple config and Xero Phase 0 run in parallel from now

D 1 5	Locked	One crypto utility — R5's key-ID envelope serves bank fields and Xero tokens
D 1 6	Locked	Xero invoices push on payment only, never on send
D 1 7	Required	Xero connection limit and certification requirements, confirmed with Xero support (X-H1)
D 1 8	Locked	App Review demo account provisioned operationally, never by a seed code path
D 1 9	Required	Account-deletion retention policy — what is deleted, what is retained, for how long (A-H1)
P10		

What was checked and how, so confidence is visible rather than implied.

Claim	Status	Basis
All 12 remediation blockers	Confirmed	Read from 9a51bd8 at the cited lines
Baseline exists as described	Confirmed	Commit present; 19 migrations incl. 0011/0014; CLAUDE.md AGENTS.md and named docs present
Controls to preserve are real	Confirmed	payment-attempt-service.ts present; 15 payment_attempts schema references
215 routes · 116 auth · 35 ownership · 71 parseInt	Confirmed	Counted in routes.ts on 9a51bd8
Zero FEATURE_* flags exist	Confirmed	grep over server/*.ts returned nothing
Test infrastructure and patterns	Confirmed	jest projects config; 23 existing tests in server/__tests__/

Xero: free Code flow, 25-connection cap	Confirmed	Xero OAuth2 documentation
Xero: certification is a security assessment	Confirmed	Xero developer-partner security requirements
Xero: tokens must be encrypted at rest, AES-128+	Confirmed	Xero security standard for API consumers
Xero: 60/min, 5,000/day, 5 concurrent, 10,000/min app-wide	Confirmed	Xero API limits documentation
Xero: refresh single-use, 60-day, 30-min grace	Confirmed	Xero token documentation and devblog
Xero: payments require AUTHORISED invoices	Confirmed	Xero invoice status documentation
Xero: Idempotency-Key on POST/PUT	Confirmed	Xero integration best practices
Xero: tiered 5-connection pricing model	Disputed	Third-party press only; contradicts Xero's docs — D17
Apple: 3.1.3(e), 4.8, 5.1.1(v), 5.1.1(ix) as cited	Confirmed	App Store Review Guidelines, fetched
Apple: privacy manifest applicability to this app	Unverified	Enforced since 12 Nov 2024, but this app may not trigger a required-reason API — one TestFlight upload settles it (A-T2)
Blank-screen behaviour on failed WebView load	Inferred	Follows from server.url with no error handler; reproduce on-device with the network off (A-T4)
Xero build: 22–32 working days	Estimated	Sums the X-T1...X-T12 task estimates

Apple config track: 6.5–10.5 working days	Estimated	Sums A-T1...A-T6 plus submission prep
Ro–R8 durations	Not estimated	The source remediation plan gives none, and Ro-T5 alone is materially larger than its siblings. Size each phase in a working session against the real baseline rather than anchoring a schedule to a guess
Tap to Pay approval lead time	Unknown	Apple does not publish it

What this supersedes

Xero & App Store Readiness · Xero Build Spec · TaptPay Master Plan · Ro Containment Tasks — all folded in here.

remains the authoritative source for Part 5 detail and its §22 phase crosswalk. Where it and this document disagree on a fact about the codebase, the verified findings in P1.2 and P10 win.

Sources — Xero

[OAuth2 overview](#) · [API limits](#) · [Scopes](#)
[Security standard for API consumers](#) · [Developer-partner security requirements](#)
[Managing tokens](#) · [Invoice status](#) · [Integration best practices](#) · [Branding](#)

Sources — Apple

[App Store Review Guidelines](#) · [Privacy manifest files](#)
[Tap to Pay — request access](#) · [Sign in with Apple](#)